

Open Source Acceleration Report for the AIGC Commerce Video Generator

Executive Summary

The fastest path for this project is **not** to hunt for a full open-source “e-commerce AI video generator” and fork it. The practical shortcut is to **compose a small number of mature, TypeScript-friendly building blocks** on top of the stack you already have: your repo already uses a monorepo layout, React, Tailwind, `@dnd-kit/*`, TanStack Query on the web side, and Fastify, BullMQ, Redis, MinIO, Swagger, and Zod on the server side. That means the highest-leverage move is to **double down on the existing choices** and only add tools that reduce bespoke UI or infrastructure work. `filecite` `turn0file0` 1

The **seven tools or patterns most worth adopting immediately** are these. **Uppy** gives you a production-grade upload UI for merchant assets with preview and resumability, instead of hand-rolling file upload UX. **React Hook Form + Zod** is the quickest way to make merchant-facing forms and keep them consistent with shared DTOs. **Keep dnd-kit and TanStack Query** because they are already in your repo and are the right primitives for sortable storyboard cards and job-status polling. **BullMQ + bull-board** gives you durable async orchestration plus an instant queue admin UI for debugging demo failures. **fastify-type-provider-zod** lets you turn Zod into request validation and OpenAPI docs in the Fastify stack you already use. **Vercel AI SDK Core** is the cleanest Node/TypeScript option for structured object generation if you can run Node 22; otherwise a thin Ark provider adapter around your existing `packages/ai` is the safer fallback. **Recharts** is the fastest dashboarding choice for the mock analytics panel. 2

The tools I would **not** introduce right now are equally important. I would avoid **LangGraph** until you genuinely need multi-agent orchestration, durable execution, or human-in-the-loop control, because your current core path is still mostly linear. I would avoid **ffmpeg.wasm** because it is explicitly experimental and materially slower than native FFmpeg. I would avoid **WebCodecs** because it is low-level and not baseline across the most widely used browsers. I would avoid **MoviePy** because it introduces a second Python runtime into a Node-first monorepo for problems you are intentionally not solving in P0/P1. I would avoid **Milvus** for now because it is built for large-scale distributed vector search and adds operational weight your three-person hackathon team does not need. I would also avoid turning **Remotion** into your P0 render engine; it is excellent for React-driven video composition, but your current product constraint is a single Seedance whole-video call, and Remotion's repo itself flags a special license that can require a company license in some cases. 3

Immediate Architecture-Fit Recommendations

For the **frontend**, your repo is already pointed in the right direction: `@dnd-kit/*` is in place for sortable interactions, TanStack Query is already installed for server-state sync, and Tailwind is already the styling base. That means you should **not** spend time swapping in Redux, XState, or a new design system during the hackathon window. Instead, add **Uppy** for the asset ingest surface, use **React Hook Form + Zod** for upload

metadata, product forms, and script-generation parameters, and keep storyboard editing intentionally simple: a reorderable list or card board first, not a full NLE timeline. For preview/export, because the current flow generates a single short video per request, a normal video player plus metadata-side editing is sufficient for P0; the value is in orchestration and merchant workflow, not local rendering complexity. 4

For the **backend**, treat **BullMQ as the async transport** and **Postgres GenerationJob as the product source of truth**. BullMQ's own docs expose a clear job lifecycle, progress updates, flows, and queue events; use those features, but persist the user-visible status, current step, retry count, and error summary in Postgres so the polling API never depends on Redis internals alone. The most practical pattern for your case is `jobId = generationJob.id`, `job.updateProgress({ step, percent, traceId })` inside processors, `QueueEvents` for lifecycle hooks, and **bounded concurrency** for remote video generation jobs so one merchant's failed run cannot stall the entire server-embedded worker. Mount **bull-board** on a protected route so the team can inspect stuck, delayed, retried, and failed jobs during the demo. 5

For **request validation and API docs**, the best fit is to move harder toward **Zod-first contracts**. Your server already depends on Fastify, `@fastify/swagger`, and Zod, and `fastify-type-provider-zod` is specifically designed to bridge Zod schemas into Fastify validation and Swagger generation. That gives you one schema source for DTOs in `packages/shared`, typed request/response contracts in `apps/server`, and optionally generated client types for the web app later. This is especially valuable because your P0 objects — `Product`, `Asset`, `GenerationJob`, `Script`, `StoryboardShot` — are exactly the sort of payloads that drift fastest when teams move quickly. 6

For **file upload and object storage**, use a two-stage approach. In **P0**, server-stream uploads through `@fastify/multipart` into MinIO/S3 are fine and low-friction; the plugin supports stream mode, per-request limits, and multipart handling in the Fastify stack you already use. In **P1**, switch to presigned uploads so large asset videos go browser-to-object-store directly, while your Node server only issues signed URLs and records metadata. Because MinIO's JS client is explicitly aimed at S3-compatible object stores, and AWS's v3 presigner has first-party docs for signed upload URLs, you can implement this without locking yourself to one vendor. 7

For **logging and observability**, do not add another logger. Fastify already uses **Pino** when logging is enabled, and Pino's own positioning is high-throughput JSON logging with low overhead. Add structured child loggers with `requestId`, `jobId`, `productId`, `merchantId`, and `traceId`; then layer **OpenTelemetry JS** later for distributed traces, because OTel is vendor-agnostic and supports automatic instrumentation in Node. For AI-specific traceability — prompt versions, model calls, parsed output, retries — **Langfuse** is the strongest open-source addition, but it is a P1 tool, not P0, because it adds another service and UI to run. 8

AI, Agent, Retrieval, and Media Decisions

Your model provider choice is the hidden advantage here: **Volcano Ark is OpenAI-compatible**, and the AI SDK ecosystem explicitly supports OpenAI-compatible providers. That means you do **not** need to adopt a heavy agent framework to get reliable structured outputs. The cleanest design is: define Zod schemas in `packages/shared`, define prompt templates and normalizers in `packages/ai`, call Ark through either **Vercel AI SDK's OpenAI-compatible provider** or a thin custom adapter, and validate every response before it becomes a `Script` or `StoryboardShot` record. One pragmatic caveat is that current AI SDK docs say

local development needs **Node 22+**; if your runtime or deployment target is still below that, the right move is not to force the upgrade for the hackathon, but to implement a narrow Ark client in `packages/ai` and keep the same schema/repair loop. ⁹

For **structured JSON reliability**, a boring workflow wins: first-pass object generation against a strict schema, immediate Zod validation, one repair pass if parsing fails, and storage of both the raw model result and the normalized parsed object for debugging. AI SDK Core supports schema-constrained object generation; LangChain JS also supports structured output with Zod and JSON Schema. In your architecture, however, **LangChain is only worth introducing when you start needing tools, retrievers, or provider-agnostic chains beyond simple request/response workflows**. Right now, script generation and storyboard generation are deterministic enough that the lighter choice is better. **LangGraph** is even farther out: its own docs position it around durable execution, memory, and human-in-the-loop agent orchestration, which matches your P2 “Agent orchestration” goals but not your P0/P1 delivery pressure. **BAML** is promising if schemas, retries, and prompt versioning become a genuine pain point, but it adds a new DSL/codegen workflow your team does not need on day one. ¹⁰

For **retrieval**, the most architecture-aligned answer is **Postgres-first**. `pgvector` lets you keep vectors in the same system of record as your products, assets, tags, and jobs, while retaining ACID semantics, joins, and ordinary Postgres operational habits. For a three-person team, that usually beats spinning up a dedicated vector service early. If you do **not** have a stable embedding model yet, you can still ship useful retrieval in P0 by storing LLM-generated or merchant-supplied tags in `jsonb`, building a generated search column, and indexing with **full-text search** and `pg_trgm` for fuzzy lookup. Postgres docs support all of those pieces directly. Move to `pgvector` in P1 when the tagging surface stabilizes. Consider **Qdrant** only when you need stronger payload filtering, hybrid lexical+dense retrieval, or when vector search starts to dominate query patterns. Avoid **Milvus** until you actually need distributed scale. Be careful with **LanceDB** in this codebase: it is attractive for embedded multimodal search and has TypeScript support, but its local-/file-oriented operational model is less aligned with your Postgres-centric system-of-record, and recent open issues around concurrent writers in the JS path are a real warning sign for a server-backed transactional app. ¹¹

For **media**, the most important conclusion is that your current product goal removes much of the reason to build a local media pipeline at all. Official Seedance materials emphasize multi-shot video generation and whole-video generation from text or image inputs, which fits your “single 12s whole-video call” constraint. So in P0/P1, focus on preview, progress, and export UX — not on FFmpeg-based composition. If you later need subtitles, BGM fallback, or lightweight muxing, add **native server-side FFmpeg** and invoke it from Node; FFmpeg’s own docs and README cover multiplexing, filtering, subtitle handling, and multiple inputs/outputs. By contrast, **ffmpeg.wasm** is still labeled experimental and its own performance docs say it will not perform as well as native FFmpeg. **WebCodecs** is powerful but very low-level and not baseline across major browsers. **MoviePy** is a respectable Python automation library, but it is the wrong runtime choice for this Node-first system unless you explicitly decide to add a media microservice later. **Remotion** should be treated as a future layer for template-driven intros/outros, subtitle-overlaid variants, or storyboard preview experiments, not as the engine you build P0 around. ¹²

Phase Recommendations

P0 should optimize for end-to-end demo success, not architectural purity. I would keep the current monorepo shape and current core stack, add **Uppy** for merchant asset upload, add **React Hook Form + Zod**

for upload/product/script forms, keep the existing **dnd-kit** board UI for storyboard ordering, keep **TanStack Query** polling for job status, formalize the server contract with **fastify-type-provider-zod**, and wire **BullMQ progress + bull-board** so everyone can see what the system is doing. On the AI side, I would use **schema-validated structured outputs** via AI SDK Core if Node 22 is available; otherwise, I would use a thin Ark client plus Zod validation and a single repair retry. On the data side, I would **not** add vectors yet unless an embedding model is already ready; use tags + Postgres FTS + `pg_trgm` first. ¹³

P1 should harden the demo path and add the features users will actually touch. That is the right time for **presigned object-storage uploads**, **pgvector** for embedding search, **Langfuse** for model-call traces and prompt versioning, **Recharts** for the mock data dashboard, and optionally a **Remotion Player** or reference-based lightweight editor view if you want richer storyboard or per-shot previews without yet owning rendering. It is also the point where **Zustand becomes optional**, but only if the creation editor develops enough cross-panel client state that component-local state becomes painful; until then, adding a global client store is probably unnecessary. ¹⁴

P2 is where you can justify additional systems because they support productionization rather than demo optics. That includes **server-side FFmpeg** for subtitle muxing and BGM fallback, **Qdrant** if retrieval semantics outgrow Postgres, **OpenTelemetry** as a broader trace/export standard, **LangGraph** if you truly implement agent orchestration or human review nodes, and **Apache ECharts** if the attribution or A/B visualization requires Sankey/funnel/advanced interaction beyond what Recharts provides quickly. This is also the only phase where a real compliance review pipeline starts making sense, because before that the bigger risk is failing to complete the generation flow at all. ¹⁵

Tool Evaluation Matrix

The table below is intentionally biased toward **tools that fit your current repo and timeline**. Where GitHub’s fetched lines did not expose a clean SPDX label, I marked the license field as **“repo LICENSE”** rather than guessing.

Name	Official source	License	Stars and maintenance	Main capability	Integration point	TS / Node / React fit	Co
Uppy	Repo / docs ¹⁶	MIT	30.8k; latest release Mar 2026	Multi-file uploads, previews, resumability, React components/headless hooks	<code>apps/web/src/features/assets</code>	Excellent for React + TS	Me

Name	Official source	License	Stars and maintenance	Main capability	Integration point	TS / Node / React fit	Co
React Hook Form	Repo / release ¹⁷	MIT	44.7k; active release May 2026	Fast forms with Zod/Yup/ AJV support	apps/web/src/features/*/forms	Excellent for React + TS	Lo
dnd-kit	Repo / release / already in repo ¹⁸	MIT	17.1k; release Apr 2026	Accessible, extensible drag/drop and sortable UIs	apps/web/src/features/creation	Excellent for React + TS	Lo
TanStack Query	Repo / release / already in repo ¹⁹	MIT	49.5k; release May 2026	Server-state caching, polling, mutations, retries	apps/web/src/lib/query, features/jobs	Excellent for React + TS	Lo
Recharts / Apache ECharts	Recharts repo / release; ECharts repo / release ²⁰	MIT / Apache-2.0	27.2k and 66.4k; both actively maintained in 2026	Dashboards, KPI cards, later advanced attribution visualizations	apps/web/src/features/dashboard	Excellent for React (Recharts easier, ECharts more powerful)	Lo Me

Name	Official source	License	Stars and maintenance	Main capability	Integration point	TS / Node / React fit	Co
BullMQ	Repo / docs / releases / architecture/ docs already in repo server deps ²¹	MIT	8.9k; release May 2026	Redis-backed queueing, retries, progress, flows, lifecycle events	apps/server/ src/jobs	Excellent for Node + TS	Me
bull-board	Repo / adapters / release ²²	repo LICENSE	3.3k; release May 2026	Queue inspector UI with Fastify adapter	apps/server/ src/admin/ bull-board.ts	Excellent for Node + Fastify	Lo
fastify-type-provider-zod	Repo / Fastify Swagger integration / release ²³	repo LICENSE	580; latest release Sep 2025	Zod request/response validation with Fastify and Swagger transforms	packages/ shared/src/ schemas , apps/server/ src/routes	Excellent for Node + TS	Lo Me
@fastify/multipart with presigned-upload upgrade path	Multipart repo / release; S3 presigner docs; MinIO JS repo ²⁴	MIT / Apache-2.0 / AWS docs	Multipart 542 stars; v10 Apr 2026; MinIO JS active with 8.0.7 Feb 2026	Stream uploads now, direct-to-object-store later	apps/server/ src/routes/ uploads.ts , services/ storage	Excellent for Node; browser uploads via signed URL	Lo Me

Name	Official source	License	Stars and maintenance	Main capability	Integration point	TS / Node / React fit	Co
Vercel AI SDK Core	Repo / structured data docs / OpenAI-compatible docs / releases ²⁵	repo LICENSE	24.4k; very active in May 2026	Provider-agnostic TS AI toolkit, structured object generation, OpenAI-compatible providers	packages/ai/src/providers, workflows	Excellent for TS/ Node if runtime is compatible	Me
OpenTelemetry JS + Langfuse	OTel repo / vendor-agnostic docs; Langfuse repo / features / releases ²⁶	repo LICENSE / repo LICENSE	3.4k + 27.7k; both active in May 2026	Distributed traces, AI observability, prompt/ version management, eval hooks	apps/server/src/observability, packages/ai/src/tracing	Strong for Node + TS	Me Hi
pgvector	Repo / Postgres release note / Postgres-compatible querying docs ²⁷	repo LICENSE	21.4k; mature low-churn extension; README currently installs v0.8.2	Vector similarity search inside Postgres, with joins and ACID semantics	apps/server/src/db, packages/shared	Excellent for Node via Postgres clients	Me
Qdrant	Repo / overview docs ²⁸	Apache-2.0	31.5k; large active repo	Vector DB with payload filtering and hybrid retrieval	infra/ later, not P0	Good TS/ Node client story	Me Hi

Name	Official source	License	Stars and maintenance	Main capability	Integration point	TS / Node / React fit	Co
Remotion	Repo / releases / docs ²⁹	Special Remotion license	47.7k; release May 2026	React-driven video composition and rendering	<code>apps/web</code> preview experiments or future render service	Strong React + TS story	Me Hi

Tools to defer or avoid for now

Name	Official source	License	Stars and maintenance	Main capability	Why not now	Complexity	Better fit to project
LangGraph	Repo / docs / releases ³⁰	MIT	2.9k; active 2026	Durable, controllable agent orchestration with memory and HITL	Your P0 flow is linear; introducing graph orchestration now adds concepts faster than value	High	Revisit in 1 agent wor or review
BAML	Repo / features / releases ³¹	Apache-2.0	8.3k; active alpha releases in May 2026	Schema-first prompt DSL with retries, streaming, generated clients	Interesting, but a new DSL/codegen loop is expensive for a hackathon-scale team	Medium to High	Keep prom TS first; br BAML only prompt es becomes unmanag

Name	Official source	License	Stars and maintenance	Main capability	Why not now	Complexity	Better fit to project
ffmpeg.wasm	Repo / perf docs / core warning ³²	MIT	17.5k; active, but still labeled experimental/ slow relative to native FFmpeg	Client-side transcode/ edit in browser	Too slow and too heavy for your current needs; adds browser CPU/ memory pain without product upside	High	Use native FFmpeg server-side only if you actually need muxing
WebCodecs	MDN / spec / compatibility note ³³	Web standard	Standardized, but low-level and not baseline in all major browsers	Browser encode/ decode primitives	Far too low-level for a three-person team shipping a demo	Very High	Use <video> for P0 and FFmpeg server-side later
MoviePy	Docs / package page ³⁴	MIT	Mature Python tool; package 2.2.1 surfaced in 2026	Python video editing automation	Wrong runtime for this monorepo; creates Python operational surface for no P0 gain	Medium to High	Stay Node.js if needed, use FFmpeg if needed
Milvus	Repo / docs / release notes ³⁵	Apache-2.0	44.4k; release May 2026	Distributed vector DB built for scale	Too much operational surface for current scale and team size	High	pgvector for Qdrant scale
LanceDB	Repo / quickstart / concurrent-writer issue evidence ³⁶	Apache-2.0	10.4k; active 2026	Embedded/ local multimodal retrieval with TS support	Attractive for local-first agent memory, but less aligned with your Postgres system-of-record and concurrency profile	Medium	Use pgvector now; revisit if you want local-first multimodal experimentation

Recommended Monorepo Integration Map and Reference Projects

The implementation mapping below is intentionally conservative: it assumes **no worker microservice split**, **no FFmpeg composition in P0/P1**, and **single whole-video generation via Seedance**. That matches your brief and the repo's current structure. turn0file0 1

- `apps/web/src/features/assets`
- `AssetUploader.tsx` : Uppy dashboard or headless uploader.
- `AssetMetadataForm.tsx` : React Hook Form + Zod resolver for tags, title, category, aspect ratio, and upload validation.
- `useCreateAssetUpload.ts` : TanStack Query mutation for create-upload/init-upload paths. 37
- `apps/web/src/features/creation`
- `StoryboardBoard.tsx` : keep dnd-kit sortable cards for shot order and simple per-shot editing.
- `ScriptEditor.tsx` : RHF-backed structured script form.
- `PreviewPanel.tsx` : regular video preview for final output; optional Remotion Player behind a feature flag when you want richer storyboard preview later. 38
- `apps/web/src/features/jobs`
- `useGenerationJobPolling.ts` : TanStack Query polling against `/generation-jobs/:id`.
- `JobProgressCard.tsx` : render `currentStep`, `percent`, `etaHint`, and retry/failure details from the API.
- Avoid SSE/WebSocket now; polling is already good enough for P0 and matches your stated plan. turn0file0 39
- `apps/web/src/features/dashboard`
- `charts/` : Recharts for P1 mock KPI charts now; reserve ECharts for P2 attribution/A-B funnels or Sankey-style visualizations. 40
- `apps/server/src/routes`
- `uploads.ts` : `@fastify/multipart` for server-stream uploads first; later add `POST /uploads/presign`.
- `generation-jobs.ts` : read Postgres-backed job status, not only Redis state.
- `openapi.ts` : register Swagger + zod transforms once. 41
- `apps/server/src/jobs`
- `queues/generation.queue.ts` : queue constructors and naming.

- `processors/generate-script.processor.ts`, `generate-storyboard.processor.ts`, `generate-video.processor.ts`: server-embedded BullMQ processors with low concurrency on video generation.
- `events/generation.events.ts`: `QueueEvents` listeners that mirror progress/failure/completion into Postgres.
- `admin/bull-board.ts`: protected queue inspector route. ⁴²
- `packages/shared/src/schemas`
- `product.ts`, `asset.ts`, `generation-job.ts`, `script.ts`, `storyboard.ts`.
- Add shared enums like `GenerationJobStep` and `GenerationJobStatus`, plus response DTOs and job payload schemas. ⁴³
- `packages/ai/src`
- `providers/ark.ts`: OpenAI-compatible Ark provider or thin fallback adapter.
- `prompts/script.ts`, `prompts/storyboard.ts`: versioned prompt builders.
- `workflows/generate-script.ts`, `workflows/generate-storyboard.ts`: structured-output + repair loop.
- `observability/tracing.ts`: no-op in P0, Langfuse hooks in P1. ⁴⁴
- `apps/server/src/search`
- `asset-search.sql.ts`: P0 tag/keyword search built on Postgres FTS + `pg_trgm`.
- `asset-vector-search.ts`: P1 pgvector query module.
- Keep tags in `jsonb`, search text in a generated/stored column, and embed later. ⁴⁵
- `infra/docker-compose.yml`
- Keep `postgres`, `redis`, `minio`.
- Add optional `langfuse` only in a P1 compose overlay so the P0 environment stays simple.
- No separate vector DB container until you have evidence Postgres is insufficient. ⁴⁶

On the “borrow architecture or components, not just ideas” question, the most useful open-source references I found were **not** full commerce-video generators, but **concrete interaction references**. `reactvideoeditor/free-react-video-editor` is a useful browser-based foundation for timeline and preview interaction patterns, but it is explicitly presented as a simplified example and should be mined for ideas rather than adopted as a hard dependency. `Storyboarder` is a much older but still instructive reference for panel metadata, animatic-style workflows, and shot-centric UX; the downside is that it is a desktop/Electron product with stale public releases, so again it is a UX reference, not a code dependency. In other words: for your project, the winning strategy is **assemble mature primitives around your existing stack**, not “fork a turnkey AI ad generator.” ⁴⁷

The final judgment is straightforward. Because Seedance already supports multi-shot whole-video generation, because Ark is OpenAI-compatible, and because your repo already contains the correct React/Fastify/BullMQ skeleton, the best technical selection is a **thin orchestration product**: uploader + schema-safe prompt workflows + queued generation + polling UI + Postgres-first retrieval. Anything more ambitious than that before the end-to-end path is solid is likely to slow the team down rather than accelerate delivery. 48

1 4 6 Bytedance-hackathon-AIGC/apps/server/package.json at main · Touricks/Bytedance-hackathon-AIGC · GitHub

<https://github.com/Touricks/Bytedance-hackathon-AIGC/blob/main/apps/server/package.json>

2 13 16 37 GitHub - transloadit/uppy: The next open source file uploader for web browsers · GitHub

<https://github.com/transloadit/uppy>

3 30 <https://github.com/langchain-ai/langgraphjs>

<https://github.com/langchain-ai/langgraphjs>

5 <https://docs.bullmq.io/guide/architecture>

<https://docs.bullmq.io/guide/architecture>

7 24 41 GitHub - fastify/fastify-multipart: Multipart support for Fastify · GitHub

<https://github.com/fastify/fastify-multipart>

8 Logging

https://fastify.io/docs/v5.4.x/Reference/Logging/?utm_source=chatgpt.com

9 <https://www.volcengine.com/docs/82379/1330626>

<https://www.volcengine.com/docs/82379/1330626>

10 <https://ai-sdk.dev/docs/ai-sdk-core/generating-structured-data>

<https://ai-sdk.dev/docs/ai-sdk-core/generating-structured-data>

11 27 <https://github.com/pgvector/pgvector>

<https://github.com/pgvector/pgvector>

12 48 <https://seed.bytedance.com/en/seedance>

<https://seed.bytedance.com/en/seedance>

14 Create a presigned URL for Amazon S3 using an AWS SDK

https://docs.aws.amazon.com/AmazonS3/latest/API/s3_example_s3_Scenario_PresignedUrl_section.html?utm_source=chatgpt.com

15 <https://ffmpeg.org/about.html>

<https://ffmpeg.org/about.html>

17 GitHub - react-hook-form/react-hook-form: React Hooks for form state management and validation (Web + React Native) · GitHub

<https://github.com/react-hook-form/react-hook-form>

18 38 GitHub - clauderic/dnd-kit: The modern toolkit for building drag and drop interfaces · GitHub

<https://github.com/clauderic/dnd-kit>

- 19 39 GitHub - TanStack/query: Powerful asynchronous state management, server-state utilities and data fetching for the web. TS/JS, React Query, Solid Query, Svelte Query and Vue Query. · GitHub
<https://github.com/TanStack/query>
- 20 40 GitHub - recharts/recharts: Redefined chart library built with React and D3 · GitHub
<https://github.com/recharts/recharts>
- 21 42 GitHub - taskforcesh/bullmq: BullMQ - Message Queue and Batch processing for NodeJS, Python, Elixir and PHP based on Redis · GitHub
<https://github.com/taskforcesh/bullmq>
- 22 GitHub - felixmosh/bull-board: Queue background jobs inspector · GitHub
<https://github.com/felixmosh/bull-board>
- 23 43 GitHub - turkerdev/fastify-type-provider-zod · GitHub
<https://github.com/turkerdev/fastify-type-provider-zod>
- 25 <https://github.com/vercel/ai>
<https://github.com/vercel/ai>
- 26 GitHub - open-telemetry/opentelemetry-js: OpenTelemetry JavaScript Client · GitHub
<https://github.com/open-telemetry/opentelemetry-js>
- 28 <https://github.com/qdrant/qdrant>
<https://github.com/qdrant/qdrant>
- 29 <https://github.com/remotion-dev/remotion>
<https://github.com/remotion-dev/remotion>
- 31 <https://github.com/boundaryml/baml>
<https://github.com/boundaryml/baml>
- 32 <https://github.com/ffmpegwasm/ffmpeg.wasm>
<https://github.com/ffmpegwasm/ffmpeg.wasm>
- 33 https://developer.mozilla.org/en-US/docs/Web/API/WebCodecs_API
https://developer.mozilla.org/en-US/docs/Web/API/WebCodecs_API
- 34 <https://zulko.github.io/moviepy/>
<https://zulko.github.io/moviepy/>
- 35 <https://github.com/milvus-io/milvus>
<https://github.com/milvus-io/milvus>
- 36 <https://github.com/lancedb/lancedb>
<https://github.com/lancedb/lancedb>
- 44 <https://ai-sdk.dev/providers/openai-compatible-providers>
<https://ai-sdk.dev/providers/openai-compatible-providers>
- 45 <https://www.postgresql.org/docs/current/textsearch-intro.html>
<https://www.postgresql.org/docs/current/textsearch-intro.html>
- 46 GitHub - langfuse/langfuse: Open source LLM engineering platform: LLM Observability, metrics, evals, prompt management, playground, datasets. Integrates with OpenTelemetry, Langchain, OpenAI SDK, LiteLLM, and more. YC W23 · GitHub
<https://github.com/langfuse/langfuse>

47 <https://github.com/sambowenhughes/a-react-video-editor>
<https://github.com/sambowenhughes/a-react-video-editor>